

Instrument Droid

Sarthak Samal^{a,1}

^aStudent

Dr. Eric Bogatin

Abstract—The Instrument Droid is a board that takes advantage of the ATmega microcontroller to drive current from an unknown VRM incrementally such that we can calculate the thevenin voltage and resistance. This board features a DAC and ADS module connected and communicating via I2C to perform mathematical operations, which gives an opportunity to compile all of our learnings from the last 3 boards and supplemental learnings to create a sort of “killer app” for our skills so far.

Contents

1	Introduction	1
2	Objectives	1
3	Plan of Record	1
3.1	What does it mean to work?	1
3.2	Key Measurements	2
3.3	Features of the Board	2
3.4	Risk Management	2
3.5	Components	2
4	SBB Version of Instrument Droid	2
4.1	Basic Concept	2
4.2	SBB Implementation and Results	3
5	PCB Design	3
6	Assembling the PCB	4
7	VRM Measuring Code	4
8	Testing	4
8.1	Power Lines and In-Rush	4
8.2	Arduino Module Testing	5
8.3	VRM Measuring Module Testing	6
8.4	Buzzer and LEDs	7
9	Conclusion	8
9.1	What Went Wrong	8
9.2	What Went Well	8
9.3	Key Takeaways	8
10	Index	8
10.1	Code	8

1. Introduction

An and important characteristic for any circuit is the thevenin voltage and resistance for the VRM it uses. However, it is rare that that information is readily available, especially custom-made or more complex VRMs. For this reason, creating a board that automatically measures thevenin characteristics for any VRMs would be immensely beneficial for circuit design. This board will make use of all that we have learned thus far to design a circuit that does just that; find the thevenin resistance and voltage of any VRM.

2. Objectives

To build the instrument droid, we will focus on two major parts. The construction of an on-board Arduino and the on-board module for measure the VRM current. To construct the on-board Arduino, we will follow the procedures for designing and verification that was used for the construction of the Golden Arduino. We will also refer

to the SBB version of the instrument droid to design and verify the on-board module.

Objectives:

1. Create the SBB version of the VRM current measuring module using an external module.
2. Design the Arduino and VRM current measuring modules in Altium Designer.
3. Solder the Arduino module and bootloader.
4. Verify the functionality of the Arduino.
5. Solder the VRM measuring module
6. Verify the communication between the ICs and the Arduino.
7. Verify the functionality of the full board by testing known VRMs.

3. Plan of Record

3.1. What does it mean to work?

The functionality of the board will be examined by confirming the functionality of each module; the Arduino and VRM measuring modules. Then, the board will be fully confirmed by running tests to confirm various key measurements.

Arduino Module:

1. Arduino must receive 5v from both the USB and power jack without complication.
2. Arduino must be able to bootload and be recognized by the Arduino IDE.
3. All power lines must experience little to no noise during operation.
4. Blink program must be able to run normally.
5. Arduino must be able to output data to the serial monitor.
6. SDA and SCL lines must be communicating with peripheral ICs.

VRM Measuring Module:

1. Sense resistor must be able to handle current as the DAC steps up current (i.e. average power dissipation is under .25W)
2. Board must be able to switch between 10Ω and 1Ω sense resistors and function normally.
3. V_{th} should be about equal to V_{load} .
4. Measured R_{th} must be about equal to a known, added R_{th} .
5. SBB results must be replicated using the newly created instrument droid.

Extra:

1. On-board LEDs must be programmable and show the progress of calculations.
2. On-board buzzer must be programmable and indicate the completion of calculations.
3. External LCD must be able to communicate via I2C lines to reliably show results.

3.2. Key Measurements

Arduino Module:

1. Tx line must send data to the Arduino
2. Tx and Rx lines must send appropriate negotiation sequences to the CH340.
3. SCL line must have a clock frequency of 100kHz and the SDA line must show some data transfer between the ATmega and peripherals.
4. USB D+ and D- readings must show clear communication between the computer and CH340 chip.
5. 16MHz crystal must oscillate at 16MHz.
6. 12MHz crystal must oscillate at 12MHz.
7. 5v must be measured on the 5v line.

VRM Measuring Module:

1. DC at MOSFET gate and sense resistor.
2. V_{th} equal to 5V VRM input.
3. R_{th} must be calculated to be around 47 Ω when 47 Ω resistor is placed in series with VRM.
4. Max current outputted by DAC must be around 26 mA

3.3. Features of the Board

1. 4-layer board (S-G-G-S)
2. 5v VCC by either USB 2.0 mini-B or 5v DC power jack.
3. 1206 components for easy soldering
4. Switches to control the power to CH340 and ATmega328p for debugging
5. Minimal test points for a more compact design.
6. SMD components for a more compact design.
7. LEDs to show progress of calculations.
8. Buzzer to indicate completion of calculations.
9. LCD display to show all necessary information without the need of an external computer.

3.4. Risk Management

1. Decoupling capacitors were added to reduce noise on power lines.
2. Ferrite bead was added to the power pin of the ATmega for a dual-pole filter.
3. Test point added on sense resistor.
4. Connectivity was checked between all pins in ICs to ensure no unexpected shorts.
5. Sub-circuits were grouped such that traces do not interfere with other sub-circuits.
6. Cross-unders were kept as few as possible to ensure one, continuous ground plane.
7. Any cross-unders that were required were done between the top and bottom signal layers, i.e. the return plane have absolutely no signal lines.

3.5. Components

1. ATmega328p x1
2. CH340p x1
3. ADS1115 x1
4. DAC (MCP4725) x1
5. TLV272 x1
6. IRL520 x1

7. 16MHz crystal oscillator x1
8. 12MHz crystal oscillator x1
9. Push button x1
10. TVS Diode
11. 5v to 3.3v LDO x1
12. Switch x4
13. 3-pin switch x3
14. 5v power jack
15. USB 2.0 mini-B connector x1
16. Red LED x5
17. 1M Ω resistor x2
18. 10k Ω resistor x5
19. 1k Ω resistor x5
20. 10ohm resistor x1
21. 1 Ω resistor x2
22. 22 μ F capacitor x6
23. 10 μ F capacitor x2
24. 1 μ F capacitor x1
25. 47nF capacitor x1
26. 22pF capacitor x4
27. 10 μ H ferrite bead x1
28. Buzzer x1
29. Smart LEDs (WS2812) x4

4. SBB Version of Instrument Droid

The solderless breadboard version of the instrument droid can be isolated to just the VRM measuring module. We will be using the same design for the Arduino module as we did for the Golden Arduino which worked as expected. Therefore, we will not comment on the SBB version of the Arduino Module.

4.1. Basic Concept

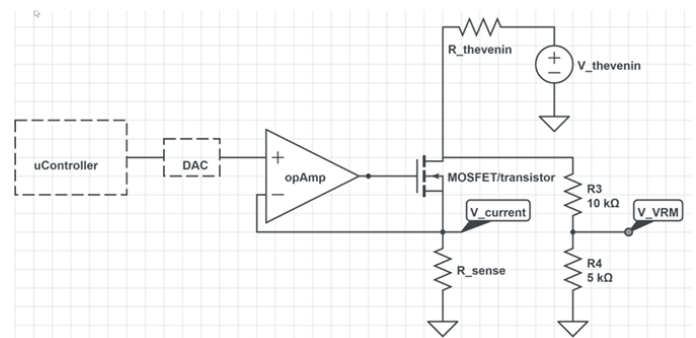


Figure 1. Schematic of the VRM Measuring Module

The module functions by taking advantage of a microcontroller (Arduino in this case), DAC and an op-amp configured as a voltage follower to incrementally activate a MOSFET. This way, we can slowly allow current through the sense resistor and get measurements quickly and slowly enough such that the resistor doesn't dissipate too much power. In the meantime, we can take advantage of a voltage divider to keep track of the VRM voltage without drawing too much current from the VRM while current flows through the sense resistor.

The current across the sense resistor will be measured by taking a differential measurement across the resistor to find the voltage drop, then a simple ohm's law calculation to find the current. Additionally, the voltage at the voltage divider will be multiplied by 3 to get an

accurate voltage measurement. All calculations will be handled by the Arduino.

4.2. SBB Implementation and Results

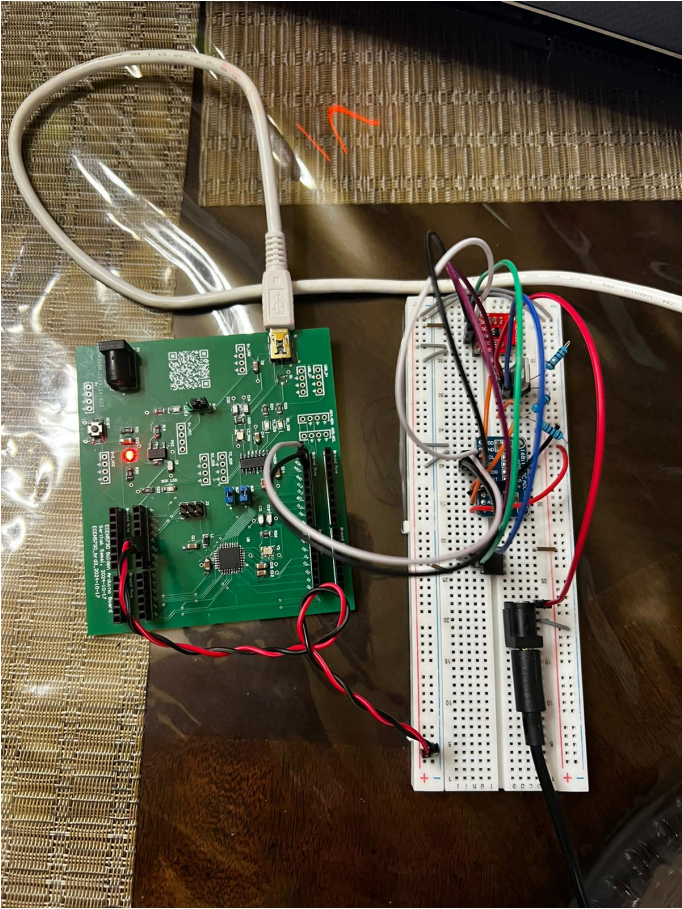
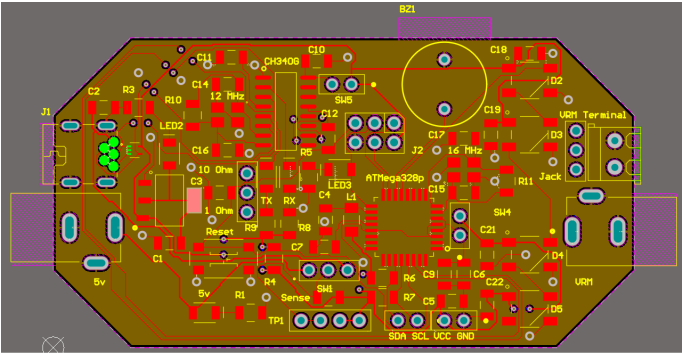
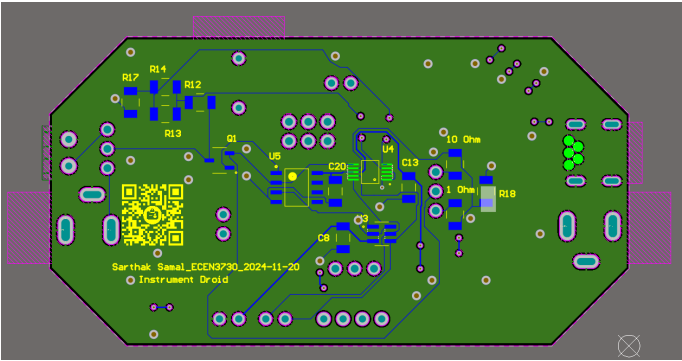


Figure 2. SBB version of VRM measuring module

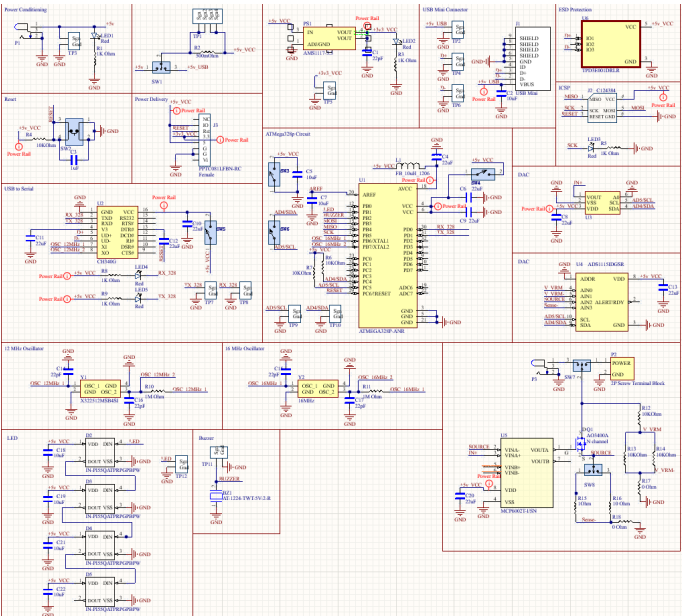
5. PCB Design



(a) Top Layer of Instrument Droid



(b) Bottom Layer of Instrument Droid



(c) Schematic of Instrument Droid

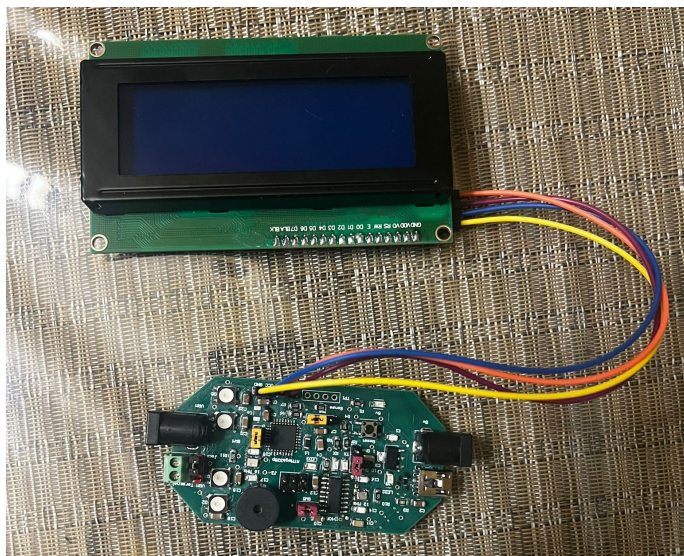
Figure 3. Schematic and Layout of Instrument Droid

When designing this board, we opt to organize all of the components such that the Arduino module is placed and routed on one side of the board while the VRM measuring module is placed on the other side. While this introduced some difficulties in routing the top layer, it allows us to easily debug each side individually.

Note: In order to keep the board compact, we opt to remove all test points. Since we know that our Arduino design works, we can be confident that (given we solder everything properly) the Arduino module will work just fine. Additionally, we can still probe each pin should we need to look at the signal behavior.

6. Assembling the PCB

Assembling the PCB required some strategy. Given the highly compact top layer, it was smartest to solder the ICs with the smaller footprint first (e.g. the ATmega and CH340 chips), before any other components. This way, we can circumvent any issues that deal with soldering in tight spaces.



(a) Unpowered instrument droid



(b) Bottom Layer of Instrument Droid

Figure 4. Powered instrument droid

After soldering, we are able to assemble the PCB as shown in fig. ?? . Before bring up and testing, we check to make sure that no unintentional shorts have occurred while soldering and after confirming this with a DMM, we move onto testing.

7. VRM Measuring Code

Please refer to the index to see the full code. The code is designed to send a pulse wave with a duty cycle of 10%, the Arduino then instructs the DAC to send a signal that increases linearly with the same duty cycle (fig. 9). For each increment, the ADS will take a differential

measurement across a sense resistor, which the Arduino will then use to calculate the thevenin voltage and resistance.

As a safety measure, the Arduino is instructed to end the program if the VRM voltage drops too low.

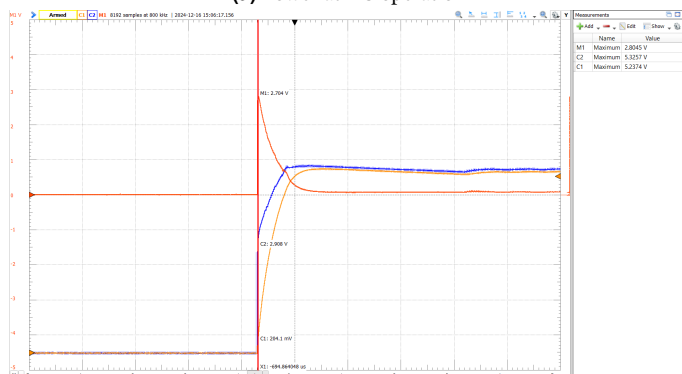
The code also sets the LEDs as a sort of progress bar to show how far the program has left to go and the buzzer to play a jingle to indicate completion.

8. Testing

8.1. Power Lines and In-Rush



(a) Power at DC operation



(b) In-rush current

Figure 5. Instrument droid power characteristics (Sense+: Blue, Sense-: Orange, Voltage across sense: Red)

Note: Channels 1 and 2 were held at an offset of -4.5v for visibility, the math channel (red) was not held at an offset.

From fig. 5, we see that the board successfully receives 5v to VCC, and that the in-rush voltage climbs to a difference of 2.8v across the sense resistor, and since we are using a 1Ω sense resistor...

$$V = IR_{sense}$$

$$2.8v = I(1\Omega)$$

$$I = 2.8A$$

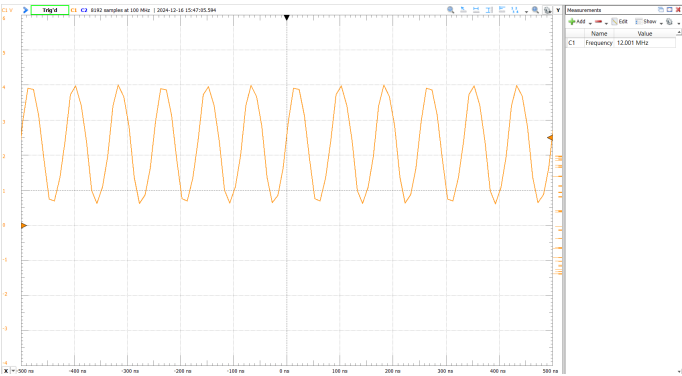
... we see that there is an in-rush current of 2.8A. 22μF capacitors were used for decoupling capacitors, however, in doing so, we create a large in-rush current. While this is not an issue for the rest of the board, this proved to be an issue for the CH340 chip since originally, a 22μF capacitor was used, creating an in-rush transient that burnt the chip. For this reason we must use a 47nF capacitor (although the datasheet specifies a 4.7nF capacitor, a 47nF one will suffice) to keep the inrush current across the chip manageable.

Under operation, the voltage and current supplied to the board remained stable, showing that the board was properly decoupled from any noise.

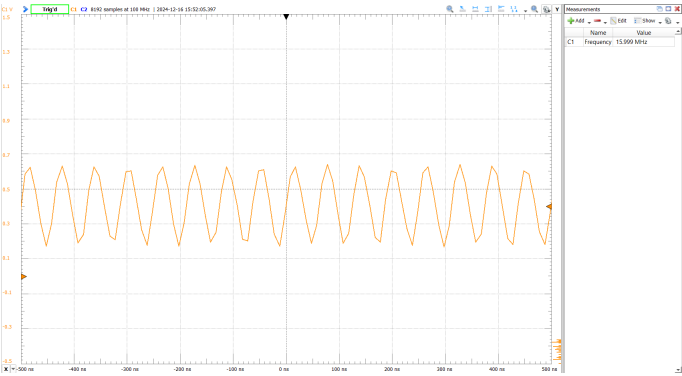
8.2. Arduino Module Testing

To test the Arudino module, we can test for two major features: quartz oscillations and data lines.

Quartz Oscillations:



(a) 12MHz crystal oscillation



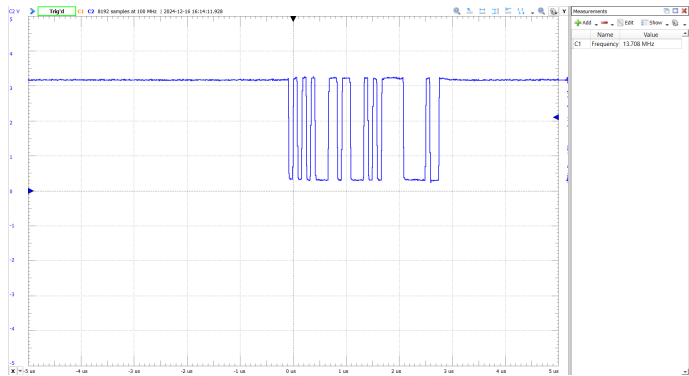
(b) 16MHz crystal oscillation

Figure 6. Crystal oscillations

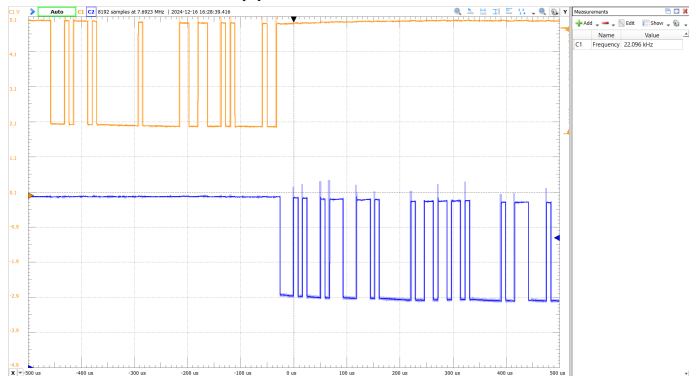
As we can see from fig. 6, both the 12MHz and 16MHz are oscillating at the proper frequencies. This shows that the ATmega chip has been successfully bootloader and can initiate communication with the computer.

Data Lines:

The data lines present on this board are the D+ and D- for the USB communication from the computer to the board, the Tx and Rx for the UART communication between the CH340 and the ATmega and finally the SDA and SCL for the I2C communication between the Arduino module and the peripherals in the VRM measuring module.



(a) USB communication



(b) UART communication

Figure 7. Crystal oscillations



(a) I2C communication

Figure 8. Serial communication in board

Note: For the sake of brevity, we will not decode the digital logic, the presence of these digital logic waveforms is enough to indicate some form of communication.

From fig. 8, we see that all of the data lines are successfully communicating with their respective peripherals and masters.

As a final test, we can try and upload a blink sketch given by Arduino LLC and see if it blinks the built in LED.



Figure 9. Blink program execution

From fig. 9, we see that the built-in LED (topmost LED) is on from the blink function (the actual blinking of the LED is difficult to see from the figure, but the LED being on is sufficient to say that the ATmega is communicating with the Arduino IDE).

With these tests, we can be sure that the Arduino module functions as expected.

8.3. VRM Measuring Module Testing

To test the VRM measuring module, we can test for four major signals/outputs: the DAC output, the MOSFET gate (see if it turns on), the current across the sense resistor, and finally, the ADS measurements.

DAC Output:

The DAC controls the opamp that drives the MOSFET. So to ensure the opamp is getting the proper signals, we check the DAC output. We should expect the signal to increase incrementally.

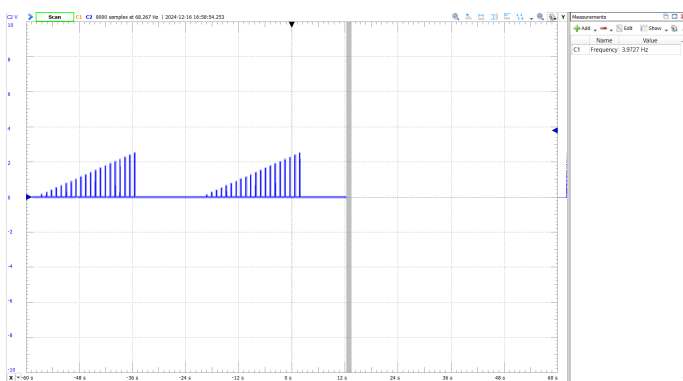


Figure 10. DAC Output

Note: The program was ran twice
From fig. 10 we see that the output of the DAC is steadily increasing from 0 to about 2.5v, which is what we expect based on the code.

MOSFET Gate Voltage:

The MOSFET is responsible for allowing the VRM to send current through the board, therefore, it is important to check if the MOSFET is working as expected. Given the opamp configuration we are working with, we should expect to see the gate voltage drop as the DAC outputs a signal to the opamp.

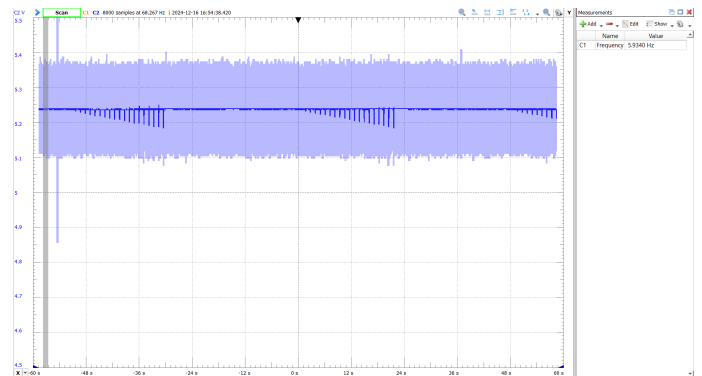


Figure 11. MOSFET gate voltage

From fig. 11, we see that the gate voltage drop in accordance with the DAC output. This signifies that the MOSFET is turning on more and more each increment, showing that our MOSFET is behaving as expected.

Sense voltage and current:

Finally, we must check the sense resistor voltage and current to see if we get what we expect. We should expect a similar waveform as the DAC: an incrementally increasing voltage at the sense resistor.

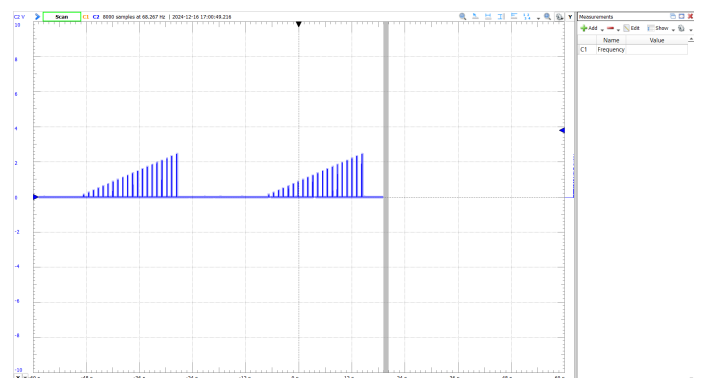


Figure 12. Voltage reading at the 10Ω resistor

From fig. 12, we find that the voltage at the sense resistor is indeed similar to the DAC output. Additionally, we should check the average voltage across the sense resistor to ensure that we do not exceed the maximum power rating. From fig. 13 we see a duty cycle of 10% with a maximum voltage of around 2.7v. Given that we are using the 10Ω sense resistor...

$$V(.1) = I_{avg}R$$

$$.27v = I_{avg}(10\Omega)$$

$$I_{avg} = .027A$$

... we will never see the average current across the sense resistor

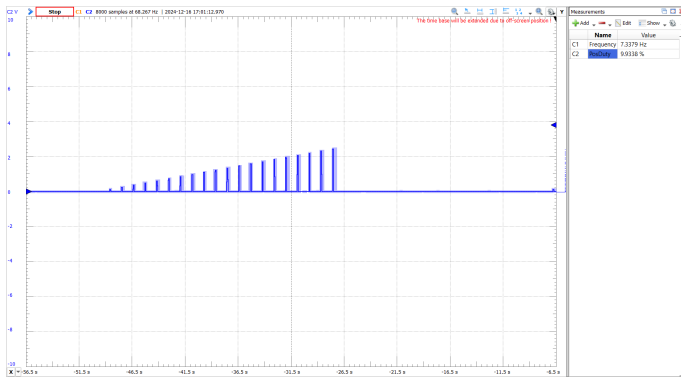
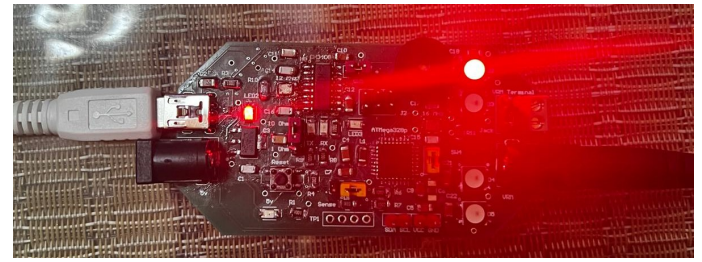
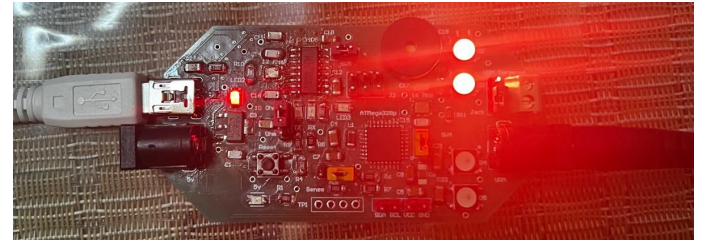


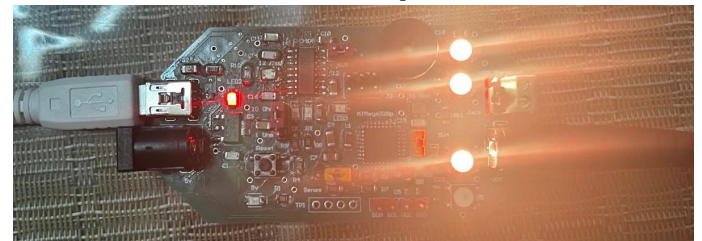
Figure 13. Sense voltage duty cycle



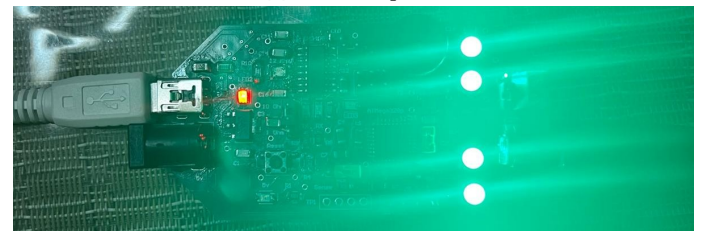
(a) LED first phase



(b) LED second phase



(c) LED third phase



(d) LED fourth phase

Figure 15. LED Phases

```

22:04:17.172 -> 1, 12.408, 5.2225, 5.2198, 0.2199
22:04:18.290 -> 2, 24.565, 5.2225, 5.2172, 0.2198
22:04:19.413 -> 3, 36.734, 5.2227, 5.2147, 0.2196
22:04:20.489 -> 4, 48.935, 5.2225, 5.2122, 0.2096
22:04:21.614 -> 5, 61.176, 5.2225, 5.2092, 0.2173
22:04:22.739 -> 6, 73.440, 5.2225, 5.2070, 0.2113
22:04:23.816 -> 7, 85.600, 5.2226, 5.2043, 0.2139
22:04:24.943 -> 8, 97.814, 5.2226, 5.2022, 0.2085
22:04:26.016 -> 9, 110.060, 5.2224, 5.2001, 0.2033
22:04:27.141 -> 10, 122.470, 5.2223, 5.1969, 0.2087
22:04:28.270 -> 11, 134.872, 5.2223, 5.1944, 0.2073
22:04:29.345 -> 12, 146.860, 5.2225, 5.1921, 0.2094
22:04:30.472 -> 13, 159.026, 5.2224, 5.1895, 0.2069
22:04:31.601 -> 14, 171.318, 5.2223, 5.1869, 0.2065
22:04:32.674 -> 15, 183.658, 5.2229, 5.1842, 0.2100
22:04:33.802 -> 16, 195.423, 5.2227, 5.1817, 0.2095
22:04:34.926 -> 17, 207.487, 5.2223, 5.1792, 0.2078
22:04:36.007 -> 18, 219.492, 5.2226, 5.1768, 0.2088
22:04:37.139 -> 19, 231.669, 5.2226, 5.1743, 0.2090
22:04:40.420 -> 20, 243.925, 5.2226, 5.1716, 0.2090

```

Figure 14. 5v VRM test

Fig. 14 shows the results from a 5v power jack. We see a different measurement for each iteration which eventually converges to a single value. This result is similar to the results from the SBB version of the instrument droid, therefore we can conclude that the ADS is measuring the voltage difference across our sense resistor properly.

8.4. Buzzer and LEDs

The buzzer and LEDs indicate the completion of the thevenin voltage and resistance calculations and the progress of calculation, respectively. We have the LEDs set to turn one depending on which iteration the Arduino is currently on, that is...

1. Phase 1 (red): iterations 1-3
2. Phase 2 (orange): iterations 4-11
3. Phase 3 (yellow): iterations 12-15
4. Phase 4 (green): iterations 16-20

The buzzer will play a series of tones (i.e. a sort of jingle) to indicate the completion of the function.



Figure 16. Buzzer Signal

From fig. 15 and 17 we see that the LED and Buzzers work as expected.

As a final test, we hook up an LCD display to show the final iteration and see if we get results consistent with fig.13.



Figure 17. Final instrument droid test

With that, we can finally conclude that our instrument droid works to calculate the thevenin voltage and resistance of any VRM!

9. Conclusion

The instrument droid is our killer app that takes advantage of our knowledge of ATmega bootloading, I2C communication and PCB practices to create a small hand-held device that can measure the thevenin voltage and resistance of any VRM plugged into the board. This board was an excellent practice for our abilities as a PCB designer as it required us to design a board such that it fit a small board size constraint and was easily solderable.

9.1. What Went Wrong

Despite coming a long way with our soldering, soldering the ADS still proved difficult. At first, there was a short between the SDA and SCL pins that was overlooked. Because of this, these lines were not able to send any communication data and couldn't communicate with the DAC, consequently, the MOSFET was not able to switch with a 10% duty cycle, thus burning our 1Ω resistor. This kept us from testing the 1Ω resistor and gathering more data to provide a more robust proof for the function of the device. However, given that there were no other points of failure in the device, we can confidently say that the 1Ω resistor would've worked. Additionally, not reviewing the datasheet for the CH340 beforehand would've saved us a chip. Had we known to use a smaller decoupling capacitor, we wouldn't have needed to replace the chip.

9.2. What Went Well

Checking for shorts with the DMM was a good habit we developed from the Golden Arduino board. Although a short between the SDA and SCL pins cost us a resistor, there was a temporary short between VCC and GND. Had that not been caught, the resulting mistake would've been much more expensive. With unintentional shorts aside, the soldering was a little easier this time around. The TVS diode (a part we left off on the Arduino due to its difficulty to solder) was easily implemented this time around.

9.3. Key Takeaways

1. Continue to check for unintentional shorts while soldering; this makes it much easier to catch large mistakes and may save hours of debugging.
2. Always review the datasheet for a component before using it. Characteristics like in-rush may matter more to some components and the only way to find out is to check the datasheet.
3. Solder the ICs and smaller footprint items first. These components may require more freedom of movement to solder properly, something that would not be available had all of the other larger components surrounding it were soldered first.
4. It is worth it to learn about certain lines and how they function (such as the Tx-Rx, SDA-SCL, D+-D- lines) so that it will be easier to debug. Due to the short on SDA-SCL, the waveform was just a clock signal at first. To the untrained eye, it may seem normal but to someone who is more familiar, something would seem wrong.

10. Index

10.1. Code

```

1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3 #include <Adafruit_MCP4725.h>
4 #include <Adafruit_ADS1X15.h>
5 #include <FastLED.h>
6 #include "pitches.h"
7
8 #define LED_PIN 9 // Specify D line for LEDs
9 #define BUZZER_PIN 10 // Dline for Buzzer
10 #define NUM_LEDS 4 // Number of LEDs in
    strip
11
12 CRGB leds[NUM_LEDS];
13
14 Adafruit_ADS1115 ads;
15 Adafruit_MCP4725 dac;
16
17 LiquidCrystal_I2C lcd(0x27, 20, 4);
18
19 float R_sense = 10.0; //
    current sensor
20 long itime_on_msec = 100; //
    on time for taking measurements
21 long itime_off_msec = itime_on_msec * 10; //
    time to cool off
22 int iCounter_off = 0; //
    counter for number of samples off
23 int iCounter_on = 0; //
    counter for number of samples on
24 float v_divider = 5000.0 / 15000.0; //
    voltage divider on the VRM
25 float DAC_ADU_per_v = 4095.0 / 5.0; //
    conversion from volts to ADU
26 int V_DAC_ADU; //
    the value in ADU to output on the DAC
27 int I_DAC_ADU; //
    the current we want to output
28 float I_A = 0.0; //
    the current we want to output, in amps
29 long itime_stop_usec; //
    this is the stop time for each loop

```

```

396 30 float ADC_V_per_ADU = 0.125 * 1e-3;          //
397     the voltage of one bit on the gain of 1
398     scale
399 31 float V_VRM_on_v;                          //
400     the value of the VRM voltage
401 32 float V_VRM_off_v;                        //
402     the value of the VRM voltage
403 33 float I_sense_on_A;                       //
404     the current through the sense resistor
405 34 float I_sense_off_A;                     //
406     the current through the sense resistor
407 35 float I_max_A = 0.25;                    //
408     max current to set for
409 36 int npts = 20;                           //
410     number of points to measure
411 37 float I_step_A = I_max_A / npts;         //
412     step current change
413 38 float I_load_A;                          //
414     the measured current load
415 39 float V_VRM_thevenin_v;
416 40 float V_VRM_loaded_v;
417 41 float R_thevenin;
418 42 int i;
419 43 int stage = 0;
420 44
421 45 int durations[] = {5,10 ,10, 5, 10, 10, 5,
422     10, 2}; // The higher the number the
423     shorter the note
424 46 int tones[] = {NOTE_B6, NOTE_C7, NOTE_A6,
425     NOTE_B6, NOTE_C7, NOTE_A6, NOTE_B6,
426     NOTE_C7, NOTE_A6};
427 47
428 48 //----LCD TEST--//
429 49
430 50 //void setup() {
431 51 // // put your setup code here, to run once:
432 52 // lcd.init();
433 53 // lcd.backlight();
434 54 //}
435 55 //
436 56 //void loop() {
437 57 // // put your main code here, to run
438     repeatedly:
439 58 // lcd.setCursor(4, 1); //Follows the format
440     : lcd.setCursor(x,y)
441 59 // lcd.print("Hello World!");
442 60 // delay(3000);
443 61 // lcd.clear();
444 62 //
445 63 // lcd.setCursor(3, 1);
446 64 // lcd.print("This is an LCD");
447 65 // lcd.setCursor(4, 2);
448 66 // lcd.print("Screen Test");
449 67 // delay(3000);
450 68 // lcd.clear();
451 69 //}
452 70
453 71 /*FUNCTIONS*/
454 72 void func_meas_off(){
455 73     dac.setVoltage(0, false);
456         //sets the output current
457 74     iCounter_off = 0;
458         //starting the current
459     counter
460 75     V_VRM_off_v = 0.0;
461         //initialize the VRM voltage
462
463         averager
464         I_sense_off_A = 0.0;
465         // initialize the current
466         averager
467         itime_stop_usec = micros() +
468         itime_off_msec * 1000;    // stop time
469
470         while (micros() <= itime_stop_usec) {
471             V_VRM_off_v = ads.
472             readADC_Differential_0_1() *
473             ADC_V_per_ADU / v_divider + V_VRM_off_v;
474             I_sense_off_A = ads.
475             readADC_Differential_2_3() *
476             ADC_V_per_ADU / R_sense + I_sense_off_A;
477             iCounter_off++;
478         }
479         V_VRM_off_v = V_VRM_off_v / iCounter_off;
480         I_sense_off_A = I_sense_off_A /
481         iCounter_off;
482         // Serial.print(iCounter_off);Serial.
483         print(", ");
484         // Serial.print(I_sense_off_A * 1e3, 4);
485         Serial.print(", ");
486         // Serial.println(V_VRM_off_v, 4);
487     }
488
489     void func_meas_on(){
490         //now turn on the current
491
492         I_DAC_ADU = I_A * R_sense * DAC_ADU_per_v
493         ;
494         dac.setVoltage(I_DAC_ADU, false); //sets
495         the output current
496
497         iCounter_on = 0;
498         V_VRM_on_v = 0.0;
499         //initialize the VRM voltage
500         averager
501         I_sense_on_A = 0.00;
502         // initialize the current
503         averager
504         itime_stop_usec = micros() +
505         itime_on_msec * 1000;    // stop time
506
507         while (micros() <= itime_stop_usec) {
508             V_VRM_on_v = ads.
509             readADC_Differential_0_1() *
510             ADC_V_per_ADU / v_divider + V_VRM_on_v;
511             I_sense_on_A = ads.
512             readADC_Differential_2_3() *
513             ADC_V_per_ADU / R_sense + I_sense_on_A;
514             iCounter_on++;
515         }
516
517         dac.setVoltage(0, false); //sets the
518         output current to zero
519
520         V_VRM_on_v = V_VRM_on_v / iCounter_on;
521         I_sense_on_A = I_sense_on_A / iCounter_on
522         ;
523         // Serial.print(iCounter_on);Serial.print
524         ("", "");
525     }

```

```

527 116 // Serial.print(I_sense_on_A * 1e3, 4);
528 Serial.print(", ");
529 117 // Serial.println(V_VRM_on_v, 4);
530 118
531 119 }
532 120
533 121 void ProgressLED(int stage) {
534 122     switch (stage){
535 123         case 1:
536 124             leds[0] = CRGB(100, 0, 0);
537 125             FastLED.show();
538 126             break;
539 127         case 2:
540 128             leds[0] = CRGB(100, 23, 0);
541 129             leds[1] = CRGB(100, 23, 0);
542 130             FastLED.show();
543 131             break;
544 132         case 3:
545 133             leds[0] = CRGB(100, 39, 0);
546 134             leds[1] = CRGB(100, 39, 0);
547 135             leds[2] = CRGB(100, 39, 0);
548 136             FastLED.show();
549 137             break;
550 138         case 4:
551 139             leds[0] = CRGB(0, 100, 0);
552 140             leds[1] = CRGB(0, 100, 0);
553 141             leds[2] = CRGB(0, 100, 0);
554 142             leds[3] = CRGB(0, 100, 0);
555 143
556 144             FastLED.show();
557 145
558 146             break;
559 147     }
560 148 }
561 149
562 150
563 151
564 152
565 153 void setup() {
566 154     lcd.init();
567 155     lcd.backlight();
568 156     Serial.begin(9600);
569 157     dac.begin(0x60); //
570     address is either 0x60, 0x61, 0x62, 0x63,
571     0x64 or 0x65
572 158     dac.setVoltage(0, false); //
573     sets the output current to 0 initially
574 159     ads.setGain(GAIN_ONE); //
575     1x gain +/- 4.096V 1 bit = 2mV
576     0.125mV
577 160     ads.begin(); //
578     note- you can put the address of the
579     ADS111 here if needed
580 161     ads.setDataRate(RATE_ADS1115_860SPS); //
581     sets the ADS1115 for higher speed
582 162
583 163     FastLED.addLeds<WS2812, LED_PIN, GRB> (leds
584     , NUM_LEDS);
585 164     fill_solid(leds, NUM_LEDS, CRGB(0, 0, 0));
586 165     FastLED.show();
587 166
588 167     pinMode(BUZZER_PIN, OUTPUT);
589 168
590 169
591 170 }
592 171
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657

```

```

658 225 //to distinguish the notes, set a
659 minimum time between them.
660 226 //the note's duration + 30% seems to
661 work well:
662 227 int pauseBetweenNotes = duration *
663 1.30;
664 228 delay(pauseBetweenNotes);
665 229
666 230 //stop the tone playing:
667 231 noTone(BUZZER_PIN);
668 232 }
669 233
670 234 }
671 235
672 236
673 237 // ----- UART Communication for each
674 increment ----- //
675 238 // Serial.print(i);
676 239 // Serial.print(", ");
677 240 // Serial.print(I_load_A * 1e3, 3);
678 241 // Serial.print(", ");
679 242 // Serial.print(V_VRM_thevenin_v, 4);
680 243 // Serial.print(", ");
681 244 // Serial.print(V_VRM_loaded_v, 4);
682 245 // Serial.print(", ");
683 246 // Serial.println(R_thevenin, 4);
684 247 }
685 248
686 249 //Serial.println("done");
687 250 while(1==1) {};
688 251
689 252 }
690

```

Code 1. Instrument Droid